

Pen testing Course 1

Rubric 1

Penetration Testing Methodology & Engagement Plan

Engagement title: Home Lab Internal Network Assessment **Document version:** 1.0

Classification: Confidential — Lab Use Only **Prepared by:** *[Your name]*, DAE Cybersecurity Program **Prepared for:** *[Lab owner / instructor]* **Date:** *[Fill in]*

Table of Contents

1. Methodology Overview (PTES)
 2. Test Plan
 3. Testing Environment
 4. Rules of Engagement (ROE)
 5. Authorization Form
 6. Scope Agreement
 7. Sample Engagement (Worked Example)
 8. Deliverables Checklist
 9. Appendix A — Tool Inventory
 10. Appendix B — References
-

1. Methodology Overview (PTES)

This engagement follows the **Penetration Testing Execution Standard (PTES)**, an industry framework that breaks an assessment into seven ordered phases. PTES is chosen over OSSTMM here because its phase structure maps cleanly onto a student engagement and produces documentation at every step — which is exactly what this project is graded on.

Think of the seven phases as a pipeline: each one produces something the next one consumes.

#	Phase	What happens	Output it produces
1	Pre-engagement Interactions	Agree scope, objectives, timeline, and legal authorization before any testing.	Signed authorization, scope agreement, ROE (Sections 4–6)
2	Intelligence Gathering	Collect information about the target — hostnames, IP addresses, exposed services.	Asset and service inventory
3	Threat Modeling	Identify what an attacker would realistically go after and how.	Prioritized list of attack surfaces
4	Vulnerability Analysis	Identify weaknesses in the discovered services.	List of candidate vulnerabilities
5	Exploitation	Confirm a vulnerability is real by safely demonstrating it.	Proof-of-concept evidence
6	Post-Exploitation	Determine the impact — what access or data the weakness exposes.	Impact assessment
7	Reporting	Document findings, risk ratings, and remediation for the client.	Final report

Key principle that runs through all seven phases: you only ever touch what you are authorized to touch. Phase 1 exists precisely so that Phases 2–6 stay legal and ethical. In a real engagement, testing outside the agreed scope — even by accident — can be a crime. In this lab you own every machine, so you are authorizing yourself, but you should still practice the discipline of writing it down.

2. Test Plan

2.1 Scope Definition

In scope (authorized targets):

Asset	Identifier	Role
Ubuntu VM	192.168.78.5 (hostname <i>Ubuntu</i> , user <i>vboxuser</i>)	Target system
Lab subnet	192.168.78.0/24 — <i>confirm with <code>ip a</code> on the VM</i>	Testing network

Out of scope (must not be touched):

- The host operating system running VirtualBox and any of your personal files.
- Any device on your home Wi-Fi that is not the lab VM (router, phones, other computers, printers, IoT devices).
- The public internet and any external services.

Testing boundaries:

- No destructive actions (no deleting data, no wiping the VM, no denial-of-service testing).
- Take a VirtualBox snapshot of the VM before testing so you can roll back.
- Testing occurs only during the scheduled window in Section 2.3.

2.2 Objectives

1. Discover which network services are running on the target VM.
2. Identify at least one exploitable or misconfigured service.
3. Demonstrate the security impact of that weakness with documented evidence.
4. Produce remediation guidance the "client" could act on.
5. Practice the full PTES workflow end to end and document every phase.

2.3 Timeline

Adjust the dates to your schedule. A realistic single-target lab engagement fits in a few sessions.

Phase	Activity	Estimated time
1	Pre-engagement: sign authorization, finalize scope	Day 1 (30 min)
2–3	Intelligence gathering + threat modeling	Day 1 (1–2 hrs)
4	Vulnerability analysis	Day 2 (1–2 hrs)
5–6	Exploitation + post-exploitation	Day 2 (1–2 hrs)
7	Reporting	Day 3 (2–3 hrs)

2.4 Deliverables

- This methodology and engagement plan (the planning artifact).
 - Signed authorization form and scope agreement.
 - Evidence log (screenshots, command output, packet captures).
 - Final penetration test report with findings, risk ratings, and remediation.
-

3. Testing Environment

Documenting the environment is itself a graded requirement, so be precise here.

3.1 Architecture

[Windows Laptop] ----- 192.168.78.0/24 lab network ----- [Ubuntu VM]

Tester machine

Target

user: mosta

192.168.78.5

user: vboxuser

3.2 Machines

Role	OS	Address	Notes
Tester	Windows (laptop)	[your laptop IP — run <i>ipconfig</i>]	Telnet client enabled

Role	OS	Address	Notes
Target	Ubuntu (VirtualBox VM)	192.168.78.5	Telnet server (<code>inetutils-inetd</code>) running on port 23

3.3 Configuration record

Document how the environment was built. You already did most of this:

- Ubuntu VM created in VirtualBox; network adapter set so the VM and laptop share the 192.168.78.0/24 subnet.
- Telnet server installed (`inetutils-telnetd`) and enabled in `/etc/inetd.conf` via `update-inetd --enable telnet`.
- `inetutils-inetd` service confirmed **active (running)** and listening on port 23 (`sudo ss -tlnp | grep :23`).
- Connectivity verified from the laptop with `ping` (ICMP) and `telnet 192.168.78.5 23` (TCP/23).

Evidence tip: the screenshots you already captured — the failed telnet attempt, the `ss` output showing port 23, and the successful login — are exactly the kind of before/after evidence a portfolio wants. Keep them.

3.4 Tools

See Appendix A for the full inventory.

4. Rules of Engagement (ROE)

Item	Agreement
Authorized tester	[Your name]
Authorized targets	192.168.78.5 only
Testing window	[dates/times from Section 2.3]
Permitted techniques	Port scanning, service enumeration, credential capture on the lab network, authenticated login as <code>vboxuser</code>

Item	Agreement
Prohibited techniques	Denial-of-service, data destruction, testing anything out of scope
Data handling	All captured data stays on the lab machines; delete or secure after the engagement
Emergency stop	If the VM becomes unstable, stop testing and restore the snapshot
Point of contact	<i>[Your name / instructor]</i>

5. Authorization Form

Why this matters: In a professional engagement, this signed form is your legal permission to test. Without it, the same actions could be unauthorized access. In your lab you are both the owner and the tester, so you sign both lines — but writing it out is the habit that keeps you on the right side of the line in a real job.

Authorization to Conduct Penetration Testing

I, the undersigned owner of the systems listed in the scope, authorize the named tester to perform security testing against the in-scope assets, during the agreed window, using the permitted techniques, subject to the Rules of Engagement above.

	Name	Signature	Date
System owner / client	_____	_____	_____
Authorized tester	_____	_____	_____

Scope reference: **192.168.78.5** (see Section 2.1). This authorization is valid only for the assets and window specified and may be revoked at any time by the system owner.

6. Scope Agreement

This section is the shared understanding both parties sign off on before work begins.

- **Assessment type:** Internal network penetration test (single host).
- **In-scope assets:** 192.168.78.5 on the 192.168.78.0/24 lab network.
- **Out-of-scope:** everything in Section 2.1's out-of-scope list.
- **Objectives:** as listed in Section 2.2.
- **Constraints:** no destructive testing; snapshot before starting; testing only in the agreed window.
- **Deliverables:** as listed in Section 2.4.
- **Acceptance:** both parties agree the scope above is complete and accurate.

Agreed by: [Client] _____ / [Tester] _____ Date: _____

7. Sample Engagement (Worked Example)

This is the demonstration of the methodology in action, walked through all seven PTES phases against your lab. The finding is the telnet service you configured.

Phase 1 — Pre-engagement

Authorization form and scope agreement (Sections 5–6) signed. Snapshot of the VM taken.

Phase 2 — Intelligence Gathering

Confirm the target is reachable and enumerate its services. A TCP port scan of 192.168.78.5 reveals **port 23/tcp (telnet) open**. Record the full service list. (*In your lab, `nmap 192.168.78.5` on a tester machine, or the `ss -tlnp` output on the VM itself, provides this inventory.*)

Phase 3 — Threat Modeling

Telnet is an unencrypted remote-access protocol. The realistic attacker goal is **credential theft**: anyone able to observe the lab network traffic can read the login and password as they cross the wire. This makes port 23 the highest-priority attack surface on the host.

Phase 4 — Vulnerability Analysis

The weakness is a **design-level insecure protocol**, not a software bug: telnet transmits the entire session — including the `vboxuser` username and password — in **cleartext**. There is no patch for this; the protocol itself is the vulnerability.

Phase 5 — Exploitation (proof of concept)

Demonstrate the impact safely by capturing a telnet login on the lab network with a packet analyzer (e.g., Wireshark) while logging in from the tester machine. The capture shows the credentials appearing as readable plaintext in the packet stream. **Evidence:** save the packet capture and a screenshot of the readable credentials. No data is altered — this is observation only, which keeps you inside the ROE.

Phase 6 — Post-Exploitation

With the captured credentials, an attacker could log in as `vboxuser` and, because that account has `sudo` rights, escalate to full administrative control of the host. Document this as the impact — you do not need to actually cause damage to establish it.

Phase 7 — Reporting

Write up the finding using a standard structure:

Finding F-001 — Cleartext Credential Transmission via Telnet Severity: High
CVSS 3.1: ~7.5 (illustrative — justify your own vector; network-adjacent, confidentiality impact high) **Affected asset:** `192.168.78.5`, port 23/tcp
Description: The host runs a telnet service that transmits authentication credentials without encryption. An attacker with access to the network segment can intercept and read valid credentials. **Evidence:** packet capture `telnet-login.pcap`; screenshot of plaintext credentials. **Impact:** Full compromise of the `vboxuser` account, which holds `sudo` privileges, leading to administrative control of the host. **Remediation:** Disable the telnet service and replace it with SSH (`sudo apt install openssh-server`), which encrypts the session. Then remove telnet: `sudo update-inetd --disable telnet` and `sudo apt remove inetutils-telnetd`. **Verification:** re-scan port 23 to confirm it is closed and confirm SSH (port 22) accepts encrypted connections.

8. Deliverables Checklist

Map each item to your rubric before you submit.

- ☐ Methodology section referencing a named standard (PTES) — Section 1
- ☐ Test plan with scope, objectives, timeline, deliverables — Section 2
- ☐ Testing environment configured and documented — Section 3
- ☐ Sample engagement using standard framework/templates — Section 7

- ☐ Authorization form — Section 5
 - ☐ Scope agreement — Section 6
 - ☐ Testing boundaries defined — Sections 2.1 and 4
 - ☐ Evidence collected (screenshots, captures, command output)
 - ☐ Final report with at least one written-up finding — Section 7 template
-

Appendix A — Tool Inventory

Tool	Purpose	Phase	Where it runs
ping	Reachability check (ICMP)	2	Tester
Nmap	Port scanning / service enumeration	2	Tester
telnet client	Connect to the target service	5	Tester (Windows)
Wireshark / tcpdump	Packet capture to demonstrate cleartext	5	Tester or VM
ss	Confirm listening ports on the target	2, verification	Target (Ubuntu)
OpenSSH	Secure replacement (remediation verification)	7	Target (Ubuntu)

Appendix B — References

- Penetration Testing Execution Standard (PTES) — <http://www.pentest-standard.org>
 - OSSTMM (Open Source Security Testing Methodology Manual) — ISECOM
 - NIST SP 800-115, *Technical Guide to Information Security Testing and Assessment*
 - CVSS v3.1 Specification — FIRST.org
-

This document is for authorized security testing in a controlled lab environment only.